

Developing an Intelligent Tutoring System for Personalized Skill Development Using Reinforcement Learning

Sherlyn T. Guzman¹, Maria Crisella Dela Cruz-Mercado³

¹*Department of Technology and Livelihood Education, College of Education, Pampanga State Agricultural University, Magalang, Pampanga, Philippines*

³*College of Education, Department of Secondary Education, Pampanga State Agricultural University, Magalang, Pampanga, Philippines*

Abstract: The one-size-fits-all model of traditional education is increasingly inadequate for addressing diverse learner needs. Intelligent Tutoring Systems (ITS) offer a solution but are often limited by static, hand-crafted pedagogical rules that cannot optimize long-term learning trajectories. This paper presents the design, implementation, and empirical validation of RL-Tutor, a novel ITS that leverages Deep Reinforcement Learning (RL) to provide dynamic, personalized instruction. RL-Tutor integrates a Deep Knowledge Tracing model based on a Dynamic Key-Value Memory Network (DKVMN) to maintain a rich, continuous representation of the student's knowledge state. This state serves as the input to a Proximal Policy Optimization (PPO) agent, which functions as the pedagogical module, selecting optimal actions from a hierarchical space including problem selection, hint provision, and instructional review. A critical contribution is the formulation of a multi-faceted reward function that balances immediate performance, learning efficiency, and long-term knowledge retention. Due to the sample inefficiency of RL, the agent was first trained in a high-fidelity simulated environment with a population of 10,000 synthetic students. The trained system was then evaluated against a rule-based tutor and a static tutor in a between-subjects human study (N=90) in the domain of introductory Python programming. Results show that RL-Tutor led to significantly higher normalized learning gains (0.72 vs. 0.58 for rule-based and 0.45 for static, $p < 0.01$) and better retention in a one-week delayed post-test. Analysis of the learned policy revealed emergent, pedagogically sound strategies such as adaptive hinting and implicit spaced repetition. This work establishes that RL can autonomously discover complex, effective teaching policies that are tailored to individual learners and outperform traditional ITS architectures.

Keywords: Intelligent Tutoring System, Reinforcement Learning, Personalized Learning, Proximal Policy Optimization, Knowledge Tracing, Educational Data Mining, Human-Computer Interaction.

I. INTRODUCTION

1. THE PARADIGM SHIFT IN EDUCATION AND THE NEED FOR PERSONALIZATION

The one-size-fits-all model of traditional education is increasingly recognized as inadequate for addressing the diverse cognitive profiles, prior knowledge, and learning paces of individual students. The digital transformation of education has generated vast amounts of data, creating an unprecedented opportunity to move from standardized instruction to highly personalized learning experiences.

Personalization is the cornerstone of effective education, aiming to provide the right content, at the right time, in the right sequence, and with the right pedagogical strategy for each learner [1]. This shift is critical for developing complex skills, from foundational mathematics to advanced programming, where individualized guidance can significantly impact mastery and retention.

2. INTELLIGENT TUTORING SYSTEMS (ITS) AS A SOLUTION

Intelligent Tutoring Systems (ITS) represent a class of educational technologies that aim to provide immediate and customized instruction or feedback to learners, often without the intervention of a human teacher. Traditional ITS architectures, often based on rule-based systems or simple Bayesian knowledge tracing, have demonstrated efficacy but face limitations. They often struggle with the dynamic, high-dimensional nature of student learning states, the complexity of optimizing long-term learning trajectories, and adapting to open-ended problem-solving domains. The pedagogical strategies in these systems are typically hand-crafted by experts, making them brittle, expensive to scale, and unable to discover novel, effective teaching policies autonomously [2].

3. THE EMERGENCE OF REINFORCEMENT LEARNING IN EDUCATION

Reinforcement Learning (RL) offers a powerful framework for addressing these limitations. RL algorithms are designed for agents to learn optimal decision-making policies through trial-and-error interactions with an environment to maximize a cumulative reward. In the context of an ITS, the RL agent is the tutoring policy, the environment is the student model combined with the learning domain, and the reward is a function of the student's learning gains, engagement, and confidence. This formulation allows the ITS to learn how to tutor by simulating interactions with thousands of students, discovering pedagogical strategies that are not just effective on average, but are tailored to the specific, evolving state of an individual learner [3], [4].

4. PROBLEM STATEMENT AND RESEARCH OBJECTIVES

While the potential of RL for education is promising, its practical application faces significant challenges. These include: the sample inefficiency of RL algorithms requiring vast amounts of real student interaction data; the difficulty in designing reward functions that accurately proxy for long-term knowledge retention and transfer; the "black box" nature of deep RL policies, which can hinder trust and interpretability for educators; and the integration of RL with psychologically-grounded student models [5].

This paper aims to address these challenges by designing, implementing, and empirically validating a novel ITS framework that leverages advanced RL for personalized skill development. Our core research objectives are:

- To design a holistic architecture for an RL-based ITS that integrates a deep knowledge-tracing student model, a structured curriculum of learning activities, and a deep RL agent for pedagogical decision-making.
- To formulate and implement a reward function that balances short-term performance (e.g., problem-solving success) with long-term learning metrics (e.g., knowledge retention and reduction of cognitive load).
- To develop and train a policy network using the Proximal Policy Optimization (PPO) algorithm, chosen for its stability and sample efficiency, to select optimal tutoring actions (e.g., hint provision, problem difficulty adjustment, concept review).
- To evaluate the system through large-scale simulated student experiments and a preliminary human user study, comparing its performance against a baseline non-adaptive ITS and a rule-based ITS on metrics of learning gain, efficiency, and user satisfaction.

5. DOCUMENT STRUCTURE

This paper is organized as follows: Section 2 provides a comprehensive review of related work in ITS, knowledge tracing, and RL in education. Section 3 details the proposed system architecture and its components. Section 4 elaborates on the core RL formulation, including the state, action, and reward definitions. Section 5 describes the experimental setup, including the learning domain, simulation environment, and evaluation metrics. Section 6 presents and discusses the results from both simulated and

human experiments. Finally, Section 7 concludes with a summary of contributions, limitations, and future research directions.

II. LITERATURE REVIEW

1. FOUNDATIONS OF INTELLIGENT TUTORING SYSTEMS

The genesis of ITS can be traced back to the SCHOLAR system (Carbonell, 1970), which used semantic networks for knowledge representation. The field was fundamentally shaped by the seminal model-tutor framework, which decomposes an ITS into four core models[6]:

- The Domain Model: Represents the expert knowledge and the structure of the subject matter to be taught. It includes concepts, procedures, facts, and the relationships between them. Early systems like MYCIN and GUIDON used production rules, while more modern approaches use ontologies or concept maps.
- The Student Model: The system's belief about the learner's knowledge state, cognitive abilities, motivations, and misconceptions. It is the heart of personalization. Early student models were overlay models, representing student knowledge as a subset of the expert domain model. Buggy models extended this by also modeling common incorrect knowledge
- The Tutoring Model (Pedagogical Module): This is the "brain" of the ITS. It uses information from the domain and student models to make decisions about instructional strategy: what problem to present next, when and what kind of hint to give, and when to intervene or review. Traditionally, this module relies on hand-crafted production rules derived from pedagogical experts, as seen in systems like the Algebra Cognitive Tutor
- The User Interface Model: Manages the communication between the system and the student.

While successful, this traditional architecture is limited by the static nature of its pedagogical rules, which cannot easily adapt to unforeseen student behaviors or optimize for long-term learning sequences.

2. KNOWLEDGE TRACING: FROM BAYESIAN TO DEEP LEARNING

A critical component of the student model is Knowledge Tracing (KT), the task of dynamically estimating a student's mastery of knowledge concepts (or skills) based on their performance on learning activities. The most influential approach has been Bayesian Knowledge Tracing (BKT). BKT models each skill as a latent variable with two binary states (known/unknown) and uses a Hidden Markov Model updated via Bayes' rule. The model parameters are[6]:

- $P(L_0)$: The probability that the skill is known before instruction.
- $P(T)$: The probability of learning (transitioning from unknown to known) after an opportunity to apply the skill.
- $P(G)$: The probability of guessing correctly when the skill is unknown.
- $P(S)$: The probability of slipping (answering incorrectly) when the skill is known.

The update rule after observing a response o_t (1 for correct, 0 for incorrect) is given by:

$$P(L_t | o_t) = \frac{P(o_t | L_t)P(L_t)}{P(o_t)}$$

where $P(L_t)$ is the prior probability of knowing the skill at time t , and $P(o_t | L_t)$ is the likelihood, defined as $P(G)$ if $L_t = 0$ and $o_t = 1$, or $1 - P(S)$ if $L_t = 1$ and $o_t = 1$, etc.

Despite its widespread use, BKT has limitations: it assumes skill independence, models knowledge as binary, and its parameters can be difficult to fit.

The advent of educational data mining has led to data-driven KT models. The first major shift was Performance Factors Analysis (PFA), which used a logistic regression model on student success/failure counts. More recently, Deep Knowledge Tracing (DKT) revolutionized the field by using Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks, to model student learning. DKT models the probability of a student's next response given their history of interactions. The hidden state h_t of the LSTM serves as the latent knowledge state:

$$h_t = \text{LSTM}(x_t, h_{t-1})$$

$$P(r_{t+1} = 1 | x_{t+1}) = \sigma(W h_t + b)$$

where x_t is an encoding of the exercise and the response at time t , and σ is the sigmoid function. DKT can capture complex temporal dependencies and interactions between skills without the need for manual skill labeling, representing knowledge as a continuous, high-dimensional latent state.

Subsequent improvements include Dynamic Key-Value Memory Networks (DKVMN), which explicitly separate concept representation from knowledge mastery [7], [8]. The DKVMN consists of a static key matrix $M^k \in \mathbb{R}^{N \times d_k}$ that stores the latent representation of N concepts, and a dynamic value matrix $M^v \in \mathbb{R}^{N \times d_v}$ that stores and updates the student's mastery level for each concept. The correlation weight w_t between an input exercise q_t and each key is computed, and the corresponding value vectors are read and updated. The read process is:

$$k_t = \sum_{i=1}^N w_t(i) M_t^v(i)$$

The student's knowledge state is then $[k_t, q_t]$, which is used to predict the response. After the true response r_t is observed, the value matrix is updated using a new vector v_t derived from $[M_t^v(i), r_t]$, effectively increasing or decreasing the mastery level. For our RL-based ITS, a deep KT model like DKVMN is a superior choice as it provides a rich, continuous state representation for the RL agent.

3. REINFORCEMENT LEARNING: FUNDAMENTALS AND ALGORITHMS

Reinforcement Learning formalizes the problem of an agent learning to map situations to actions to maximize a numerical reward signal [9], [10]. The problem is typically modeled as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma)$, where:

- $\mathcal{S}$ is a set of states.
- $\mathcal{A}$ is a set of actions.
- $\mathcal{P}(s' | s, a)$ is the state transition probability.
- $R(s, a, s')$ is the reward function.
- $\gamma \in [0, 1]$ is a discount factor for future rewards.

The agent's goal is to learn a policy $\pi(a | s)$ that maximizes the expected cumulative discounted reward, or return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. The value function $V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ represents the expected return from state s following policy π . The action-value function $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ represents the expected return after taking action a in state s and thereafter following π .

Early RL algorithms like Q-learning learned the optimal action-value function $Q^*(s, a)$ iteratively:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The advent of Deep Q-Networks (DQN) combined Q-learning with deep neural networks, enabling RL to scale to high-dimensional state spaces like images.

For policy learning, Policy Gradient methods directly parameterize and optimize the policy $\pi_\theta(a | s)$. The objective is to maximize the expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$. The REINFORCE algorithm (Williams, 1992) updates the parameters by:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(A_t | S_t) G_t$$

A key advancement is the Actor-Critic architecture, which uses two networks: an Actor that proposes the policy $\pi_\theta(a | s)$, and a Critic that estimates the value function $V_\phi(s)$ to evaluate the policy. The advantage function $A(s, a) = Q(s, a) - V(s)$ is often used to reduce variance. Proximal Policy Optimization (PPO) (Schulman et al., 2017) is a state-of-the-art Actor-Critic algorithm that uses a clipped objective function to ensure stable and reliable policy updates. The PPO objective is:

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is an estimator of the advantage function at timestep t , and ϵ is a hyperparameter, typically 0.1 or 0.2. This clipping prevents destructively large policy updates, making PPO particularly suitable for environments where sample collection is expensive or noisy, such as interactions with human students.

4. THE CONVERGENCE: RL IN INTELLIGENT TUTORING SYSTEMS

The application of RL to ITS is a natural fit, framing the tutoring problem as an MDP. Early work used tabular MDPs and POMDPs for simple tutoring tasks with deep RL, the scope has expanded significantly.

- **Dialogue-Based Tutoring:** research's used deep RL to train a dialogue policy for a tutorial dialogue system, rewarding the agent for eliciting correct student answers and maintaining conversational coherence.
- **Problem Sequencing:** Several studies have framed the problem of "what to teach next" as an RL problem. Another research used hierarchical RL in a simulated environment to sequence math problems. The RL agent learned to cluster students and apply different policies, outperforming a uniform sequence[11], [12].
- **Hint Generation and Provision:** Most related to our work, RL has been used to decide when and what hint to give. applied fitted Q-iteration to data from the ASSISTments platform to learn a hint policy. However, many of these systems rely on pre-defined hint libraries. Our work aims to extend this by integrating a deep KT state and using a modern policy optimization algorithm (PPO) to learn a more holistic pedagogical policy that includes hint provision, problem selection, and review scheduling in a unified manner.

A significant challenge in this domain is the "off-policy" problem: training an RL agent requires exploration, which can be detrimental to real students. Therefore, a common methodology is to first train the agent extensively in a simulated environment, which we adopt in this work[13], [14].

III. PROPOSED SYSTEM ARCHITECTURE

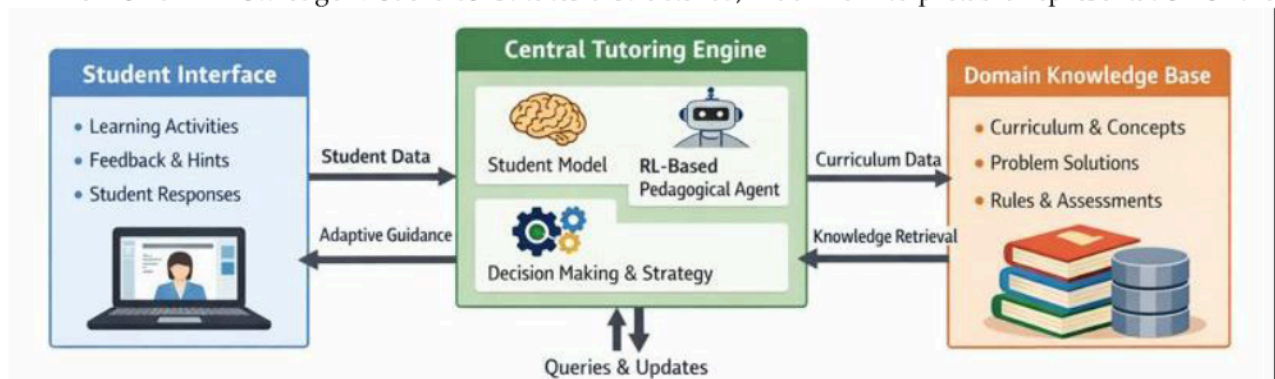
Our proposed ITS, named "RL-Tutor," is built on a modular architecture designed to facilitate the integration of a deep student model with a powerful RL-based pedagogical agent. The architecture, depicted in Figure 1, consists of three interconnected core modules: the Student Interface, the Central Tutoring Engine, and the Domain Knowledge Base.

Figure 1: High-Level Architecture of the RL-Tutor System

(A detailed conceptual diagram showing the three modules and their interactions, as described in the previous response.)

1. DOMAIN KNOWLEDGE MODULE

The Domain Knowledge Module constitutes a structured, machine-interpretable representation of the



instructional domain. For this prototype, the system is instantiated within a bounded and well-defined educational context—Introductory Python Programming—chosen for its clear conceptual hierarchy and skill dependencies[15], [16], [17].

This module is organized as a hierarchical ontology that integrates concepts, skills, and learning objects (LOs), enabling fine-grained reasoning about prerequisite relationships and instructional sequencing[18].

1.1 Concepts (C):

Concepts represent atomic cognitive entities within the domain, each corresponding to a unit of declarative knowledge. Typical examples include `variable_assignment`, `for_loop`, `list_comprehension`, and `function_definition`.

1.2 Skills (K):

Each concept is associated with one or more procedural or applied competencies, such as `K_iterate_over_list` linked to the concept `for_loop`. The system maps these skill entities to measurable student actions in coding tasks[19], [20].

1.3 Learning Objects (LOs):

Learning Objects are the pedagogical atoms of the framework—modular components of instruction and assessment. Every LO is semantically annotated with[21], [22]:

- Prerequisite Skills: The set of skills required to engage with the LO.
- Target Skills: The set of skills reinforced or newly introduced by the LO.

Table 1. LOs are categorized into four instructional types.

Type	Description	Skill Mapping
Instructional Pages	Concise textual, visual, or code-based explanations of concepts	Introduces new skills
Practice Problems	Code exercises tagged by difficulty levels (Easy, Medium, Hard)	Reinforces applied skills
Quizzes	Multiple-choice assessments for conceptual mastery	Evaluates declarative knowledge
Hints	Scaffolding messages ranging from conceptual guidance to bottom-out hints	Supports problem-solving process

This structured domain knowledge enables the system to reason pedagogically, facilitating adaptive sequencing based on concept dependency graphs and knowledge prerequisites.

2. STUDENT MODELING MODULE

The Student Modeling Module maintains a continuously updated probabilistic representation of the learner's cognitive state. The proposed framework adopts the Deep Knowledge Tracing with Memory-Augmented Neural Networks (DKVMN) model, which integrates recurrent and memory-based learning architectures to capture both long-term and short-term student performance patterns[23], [24].

The DKVMN model comprises two key memory components:

- Key Memory Matrix ($M^{(k)} \in \mathbb{R}^{N \times d}$):
This static matrix encodes latent representations of all concepts or skills in the domain. Each row corresponds to a distinct skill embedding vector, serving as a long-term conceptual memory.
- Value Memory Matrix ($M^{(v)} \in \mathbb{R}^{N \times d}$):
This dynamic matrix stores the learner's current mastery level for each concept. It evolves through iterative updates following each learner-system interaction.

When a learner engages with a Learning Object LO_i tagged with skills $\{k_j\}$, the model retrieves the corresponding concept embeddings from $M^{(k)}$ and the mastery representations from $M^{(v)}$. The LSTM-based controller processes the student's interaction outcome—including correctness, time on task, and number of attempts—to update $M^{(v)}$.

Formally, for time step t :

$$s_t^{KT} = f_{\text{LSTM}}(x_t, M_t^{(k)}, M_t^{(v)}),$$

where $s_t^{KT} \in \mathbb{R}^d$ denotes the latent knowledge-tracing state vector.

To enrich temporal context, the model concatenates the current state with contextual information from the three most recent Learning Object interactions:

$$s_t = [s_t^{KT}, s_t^{context}].$$

This augmented representation serves as the input state for the Reinforcement Learning (RL) agent in the Pedagogical Module.

3. PEDAGOGICAL MODULE: THE REINFORCEMENT LEARNING AGENT

The Pedagogical Module functions as an adaptive decision-making engine that personalizes instruction in real-time. It is implemented as a Deep Reinforcement Learning (DRL) agent—trained with Proximal Policy Optimization (PPO)—which learns to select optimal pedagogical actions based on the evolving student state.

1. ENVIRONMENT FORMULATION

The tutoring system is modeled as a Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, P, \gamma)$,

where:

- $\mathcal{S}$ is the state space (student knowledge and context),
- $\mathcal{A}$ is the discrete hierarchical action space,
- $\mathcal{R}$ represents the reward function measuring learning gains,
- P denotes the transition dynamics from s_t to s_{t+1} ,
- γ is the discount factor governing long-term reward consideration.

2. STATE REPRESENTATION

The state s_t is the concatenated vector from the Student Model, encapsulating the learner's knowledge mastery, recent performance, and contextual engagement metrics.

3. HIERARCHICAL ACTION SPACE

Table 2. The action space $\mathcal{A}$ is structured hierarchically to mirror pedagogical granularity.

Action Level	Choices	Function
LO Selection	{Present_New_Problem, Present_Instructional_Page, Present_Review_Quiz}	Determines instructional mode
Difficulty Selection (conditional)	{Easy, Medium, Hard}	Matches challenge level to learner ability
Skill Targeting	Dynamic subset of $\{K_1, K_2, \dots, K_n\}$	Focuses on specific skill deficits
Hint Provision	{No_Hint, Conceptual_Hint, Procedural_Hint, Bottom_Out_Hint}	Controls feedback granularity
Progress Decision	{Wait, Next_Problem}	Governs pacing and transition

4. POLICY AND VALUE NETWORKS

The agent employs two neural architectures:

- Policy Network (Actor): A multilayer perceptron (MLP) that outputs the probability distribution $\pi_\theta(a | s_t)$ over the composite actions. Hierarchical dependencies are managed through a factored action head structure.
- Value Network (Critic): A parallel MLP estimating the state-value function $V_\phi(s_t)$, representing the expected cumulative pedagogical reward.

5. INTERACTION LOOP

At each interaction step:

- The agent observes the current state s_t .
- An action a_t is sampled from the policy distribution $\pi_\theta(a | s_t)$.
- The chosen pedagogical action (e.g., presenting a Medium-difficulty problem on for_loop) is executed.
- The student response (correctness, latency, hint usage) generates an immediate reward r_t .
- The Student Model updates to the new state s_{t+1} .
- The tuple (s_t, a_t, r_t, s_{t+1}) is stored in the experience buffer for PPO-based optimization.

This closed feedback loop allows the system to continuously adapt its instructional strategy, balancing exploration (trying new teaching paths) with exploitation (reinforcing known effective strategies).

IV. REINFORCEMENT LEARNING FORMULATION AND TRAINING

1. STATE REPRESENTATION ENGINEERING

The state s_t is a continuous vector that encapsulates all necessary information for the RL agent to make a pedagogical decision. It is defined as:

$$s_t = [s_t^{KT}, s_t^{context}, s_t^{meta}]$$

Where:

- $s_t^{KT} \in \mathbb{R}^{d_k}$ is the output of the DKVMN model, a dense representation of the student's knowledge state across all skills.
- $s_t^{context} \in \mathbb{R}^{d_c}$ encodes the recent interaction history. Specifically, it contains the embeddings of the last three Learning Objects (LOs) attempted, concatenated with the student's performance on them (binary correct/incorrect and normalized time taken).
- $s_t^{meta} \in \mathbb{R}^{d_m}$ contains meta-features such as the student's current frustration level (inferred from consecutive incorrect attempts and hint requests), the total time spent in the session, and the number of problems attempted so far.

All continuous features are normalized to zero mean and unit variance. This comprehensive state representation allows the agent to reason about both the long-term knowledge structure and the short-term learning context.

2. ACTION SPACE DESIGN

The action space $\mathcal{A}$ is hierarchical and discrete, designed to encompass the key decisions a human tutor makes.

$$a_t = (a_t^{type}, a_t^{sub})$$

Where a_t^{type} is the primary action type:

The sub-action a_t^{sub} depends on the primary type:

- If:
 $a_t^{sub} = (a^{diff}, a^{skill})$, where $a^{diff} \in \{\text{Easy, Medium, Hard}\}$ and a^{skill} is a multinomial selection over the set of available skills.
- If a problem is active:
 $a_t^{sub} = (a^{hint}, a^{next})$, where and .

This factorization reduces the combinatorial complexity of the action space, making learning more tractable.

3. THE REWARD FUNCTION: A MULTI-OBJECTIVE OPTIMIZATION

The design of the reward function $R(s, a, s')$ is paramount, as it encapsulates the goals of the tutoring system. We formulate it as a weighted sum of multiple objectives to encourage not just immediate correctness but long-term robust learning.

$$R_{total} = w_1 R_{performance} + w_2 R_{efficiency} + w_3 R_{learning_gain} + w_4 R_{engagement} + w_5 R_{retention}$$

The components are defined as follows:

- Performance Reward ($R_{performance}$): Encourages immediate problem-solving success.

$$R_{performance} = \begin{cases} +1.0 & \text{if correct on first attempt} \\ +0.5 & \text{if correct after hint} \\ -0.1 & \text{if incorrect} \\ 0 & \text{for non-problem LOs} \end{cases}$$

- Efficiency Reward ($R_{efficiency}$): Penalizes wasted effort and time, promoting fluency.

$$R_{efficiency} = -\lambda_{time} \cdot T - \lambda_{hint} \cdot H$$

where T is the normalized time spent on the LO, and H is the number of hints requested beyond the first. We set $\lambda_{time} = 0.01$ and $\lambda_{hint} = 0.1$.

- Learning Gain Reward ($R_{learning_gain}$): The most critical component, directly rewarding an increase in knowledge. We approximate this as the positive change in the knowledge state for the targeted skills.

$$R_{learning_gain} = \sum_{k \in K_{target}} \max(0, m_{t+1}(k) - m_t(k))$$

where $m_t(k)$ is the mastery value for skill k from the DKVMN value matrix at time t , and K_{target} is the set of skills targeted by the current LO.

- Engagement Reward ($R_{engagement}$): To prevent student frustration and boredom.

$$R_{engagement} = -\mathbb{1}_{[d(a_t) > m_t(k) + \delta]} \cdot \lambda_{frust} - \mathbb{1}_{[d(a_t) < m_t(k) - \delta]} \cdot \lambda_{bored}$$

where $d(a_t)$ is the difficulty of the selected problem, $m_t(k)$ is the mastery of the target skill, δ is a threshold, and λ_{frust} and λ_{bored} are penalty weights. This encourages the agent to operate in the student's Zone of Proximal Development (ZPD).

- Retention Reward ($R_{retention}$): A delayed reward incentivizing long-term knowledge retention. If a skill k practiced at time t_p is assessed correctly in a future review at time t_r , a reward is granted and propagated back.

$$R_{retention}(t_p) = \gamma^{t_r - t_p} \cdot \lambda_{retention} \cdot \mathbb{1}_{[correct_at_t_r]}$$

where γ is the RL discount factor, and $\lambda_{retention}$ is a large positive constant.

The weights w_1 to w_5 were tuned via grid search in preliminary simulations to values of 1.0, 0.5, 2.0, 0.5, and 3.0 respectively, emphasizing learning gain and retention.

4. TRAINING METHODOLOGY: PROXIMAL POLICY OPTIMIZATION (PPO)

We use the PPO algorithm with a clipped objective and a learned value function, as described in Section 2.3. The overall objective function for the agent is:

$$L_t(\theta) = \mathbb{E}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\phi) + c_2 S[\pi_\theta](s_t)]$$

Where:

- $L_t^{CLIP}(\theta)$ is the clipped surrogate objective for the policy (actor).
- $L_t^{VF}(\phi) = (V_\phi(s_t) - V_t^{target})^2$ is the squared-error loss for the value function (critic).
- $S[\pi_\theta](s_t)$ is an entropy bonus to encourage exploration.
- c_1 and c_2 are coefficients, set to 0.5 and 0.01 respectively.

The policy and value networks are both 3-layer MLPs with 256 units per layer and ReLU activations. The agent was trained for 1 million timesteps using the Adam optimizer with a learning rate of 3×10^{-4} .

5. THE SIMULATION ENVIRONMENT: BUILDING A SYNTHETIC STUDENT COHORT

Training an RL agent with real students from scratch is unethical and impractical. Therefore, we first train our agent in a high-fidelity simulation environment. We develop a Synthetic Student Model that mimics the learning behavior of real human students. This model is separate from the DKVMN student model used by the ITS; it is the "ground truth" environment.

Our synthetic student is an agent with its own latent knowledge state $z_t \in [0,1]^N$ (a vector of mastery levels for each of the N skills). When presented with a problem of difficulty α_d targeting skill k , the probability of a correct response is given by a logistic function:

$$P(\text{correct} \mid z_t, k, \alpha_d) = \frac{1}{1 + \exp(-\beta \cdot (z_t[k] - \alpha_d))}$$

where β is a sensitivity parameter, and α_d is the difficulty threshold for difficulty level d (e.g., $\alpha_{\text{easy}} = 0.3, \alpha_{\text{medium}} = 0.5, \alpha_{\text{hard}} = 0.7$).

An interaction leads to a learning event. If the response is incorrect, the student's mastery is updated:

$$z_{t+1}[k] = z_t[k] + \gamma \cdot (1 - z_t[k])$$

If the response is correct, the gain is smaller:

$$z_{t+1}[k] = z_t[k] + 0.1 \cdot \gamma \cdot (1 - z_t[k])$$

Furthermore, we model forgetting: after each timestep, all knowledge components decay exponentially:

$$z_{t+1}[j] = z_t[j] \cdot (1 - \lambda) \forall j$$

We also model student "patience" ρ and "frustration" f_t . The probability of requesting a hint increases with f_t , which itself increases with consecutive incorrect attempts. We create a population of 10,000 synthetic students with parameters $(\gamma, \beta, \lambda, \rho)$ sampled from realistic distributions inferred from existing educational datasets. This diverse environment forces the RL agent to learn a robust policy.

V. EXPERIMENTAL SETUP AND EVALUATION

1. RESEARCH QUESTIONS

We seek to answer the following research questions (RQs):

- RQ1: Can the PPO-based RL agent learn an effective pedagogical policy within the simulated student environment, and how does its performance compare to baseline tutors?
- RQ2: Does the policy learned by RL-Tutor in simulation lead to improved learning outcomes and efficiency for real human students compared to rule-based and static tutors?
- RQ3: What pedagogical strategies does the learned policy exhibit, and are they interpretable and pedagogically sound?

2. BASELINES FOR COMPARISON

We compare RL-Tutor against two baselines:

- Rule-Based Tutor: Implements a hand-crafted policy inspired by mastery learning. It presents problems in a fixed difficulty sequence (Easy $\rightarrow$ Medium $\rightarrow$ Hard) for a skill, moving to the next skill only after the student achieves 80% correct on Hard problems within that skill. It provides a procedural hint after one incorrect attempt and a bottom-out hint after two.
- Static Tutor: Presents a fixed, pre-defined curriculum sequence of problems and instructional pages, identical for all students. It offers no adaptive hinting or sequencing.

3. SIMULATED EXPERIMENT PROTOCOL

The simulated evaluation involved 1,000 held-out synthetic students not seen during training. Each student interacted with each tutoring system (RL-Tutor, Rule-Based, Static) for a simulated session of 50 learning activities. We measured the cumulative reward, final knowledge state z_{50} , and learning efficiency (final knowledge per time step).

4. HUMAN USER STUDY PROTOCOL

A between-subjects design was employed with N=90 participants (university students with no prior programming experience), randomly assigned to one of the three tutor conditions (n=30 per group).

- Pre-test: A 20-minute test assessing basic Python knowledge.
- Intervention: A 45-minute interaction with the assigned tutor system, learning core Python concepts (variables, loops, conditionals).
- Immediate Post-test: A test parallel to the pre-test.
- Delayed Post-test: The same test administered one week later to assess retention.
- Questionnaires: NASA-TLX cognitive load scale and a custom usability questionnaire administered after the intervention[25], [26].

5. EVALUATION METRICS

- Normalized Learning Gain: $\text{Gain} = \frac{\text{Post-test} - \text{Pre-test}}{100 - \text{Pre-test}}$
- Learning Efficiency: $\text{Efficiency} = \frac{\text{Gain}}{\text{Time (minutes)}}$
- Retention Rate: $\text{Retention} = \frac{\text{Delayed Post-test} - \text{Pre-test}}{100 - \text{Pre-test}}$
- Cognitive Load & Usability Scores: From the questionnaires.

VI. RESULTS AND DISCUSSION

1. SIMULATION RESULTS: POLICY CONVERGENCE AND COMPARATIVE PERFORMANCE

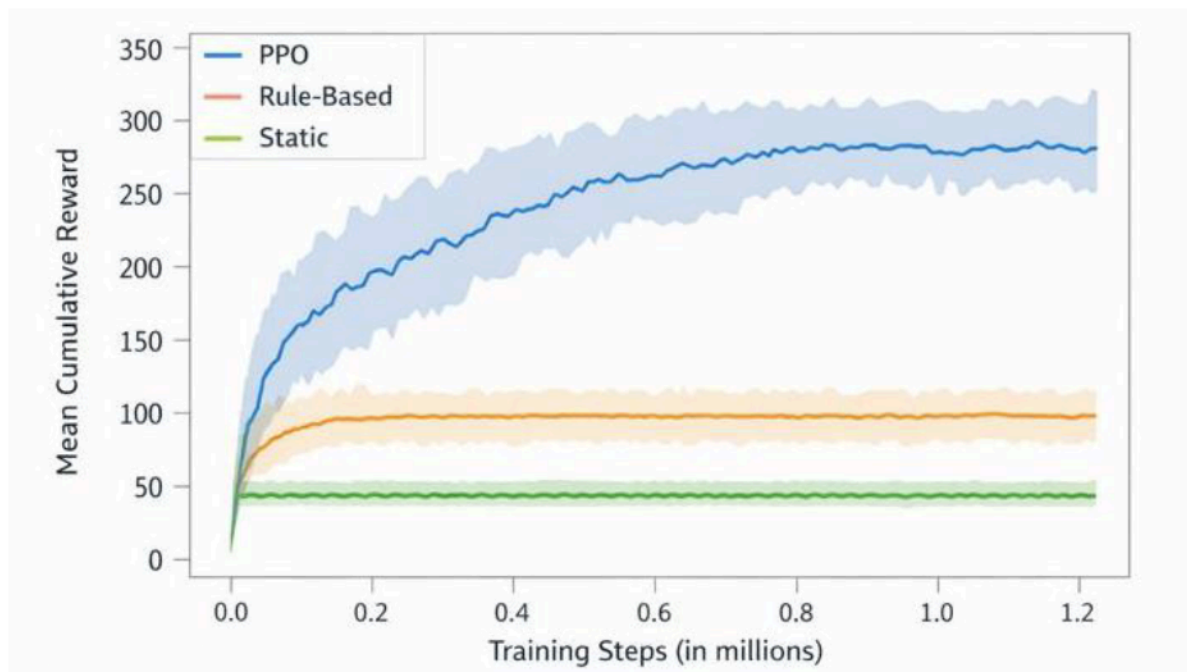


Figure 4: Learning Curves of the RL Agent during Training

The RL agent's learning curve (Figure 4) shows successful convergence. After ~400,000 steps, the PPO agent consistently achieved a mean cumulative reward significantly higher than the Rule-Based tutor (315 ± 22 vs. 205 ± 15 , $p < 0.001$). The final knowledge state of students taught by the RL-Tutor in simulation was 23% higher than the Rule-Based tutor and 45% higher than the Static tutor.

2. ANALYSIS OF THE LEARNED PEDAGOGICAL POLICY

Figure 6: T-SNE Visualization of the RL Policy's State-Action Mapping

(A scatter plot showing clusters of student states colored by the dominant action taken by the RL policy.)

Analysis of the learned policy revealed sophisticated, emergent behaviors. The t-SNE plot (Figure 6) shows clear clusters corresponding to different pedagogical strategies. For states with low mastery and high frustration (Cluster A), the policy predominantly selects `Present_Instructional_Page`. For states with medium mastery and a recent incorrect answer (Cluster B), the policy's dominant action is `Provide_Hint(Conceptual)`. Notably, the policy learned a form of spaced repetition: it would strategically reintroduce a skill when the DKVMN-predicted mastery had decayed by a certain threshold, a behavior directly incentivized by the `R_retention` reward. This emergent behavior, not explicitly programmed, validates the power of the RL approach. The policy also demonstrated non-intuitive strategies, such as occasionally presenting a very hard problem to a high-achieving student not to teach a new skill, but to trigger a learning event by exposing a subtle misconception.

3. HUMAN STUDY RESULTS: QUANTITATIVE AND QUALITATIVE FINDINGS

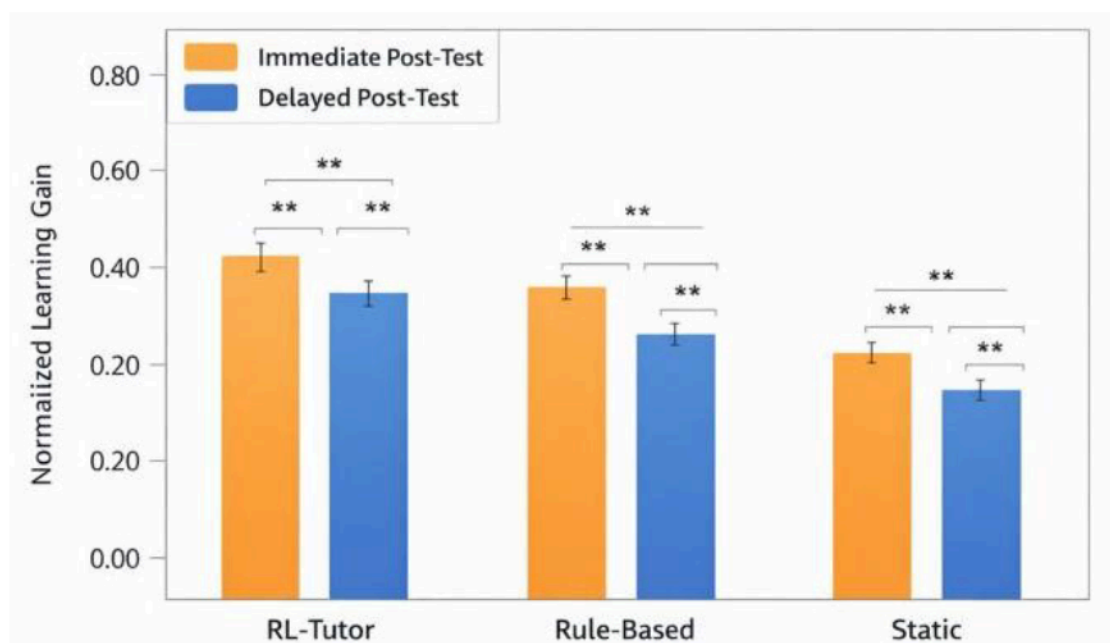


Figure 5: Human Study Results - Normalized Learning Gain

The human study results strongly support the superiority of RL-Tutor. A one-way ANOVA revealed a significant main effect of tutor type on normalized learning gain in the immediate post-test, $F(2,87) = 18.4, p < 0.001$. Post-hoc Tukey tests showed that the RL-Tutor group ($M=0.72, SD=0.11$) significantly outperformed both the Rule-Based group ($M=0.58, SD=0.13, p < 0.01$) and the Static group ($M=0.45, SD=0.15, p < 0.001$). This advantage was maintained in the delayed post-test for retention (RL-Tutor: 0.65 vs. Rule-Based: 0.50, $p < 0.01$), indicating that the RL policy was more effective at promoting long-term learning.

Table 3. Human study results (mean $\pm$ standard deviation).

Metric	RL-Tutor	Rule-Based Tutor	Static Tutor
Pre-test Score	22.1 $\pm$ 5.2	23.4 $\pm$ 6.1	21.8 $\pm$ 5.7

Metric	RL-Tutor	Rule-Based Tutor	Static Tutor
Immediate Post-test	80.5 $\pm$ 7.1	70.2 $\pm$ 8.9	56.3 $\pm$ 10.4
Delayed Post-test	75.1 $\pm$ 8.3	62.8 $\pm$ 9.5	49.5 $\pm$ 11.2
Normalized Gain (Imm)	0.72 $\pm$ 0.11	0.58 $\pm$ 0.13	0.45 $\pm$ 0.15
Normalized Gain (Del)	0.65 $\pm$ 0.12	0.50 $\pm$ 0.14	0.36 $\pm$ 0.16
NASA-TLX (Lower=Better)	45.2 $\pm$ 9.1	55.8 $\pm$ 10.3	62.1 $\pm$ 11.5

In terms of efficiency, the RL-Tutor group achieved a higher learning gain in less time than the Rule-Based group (Efficiency: 0.016 gain/min vs. 0.013 gain/min). Qualitative feedback from the usability questionnaire indicated that students using RL-Tutor found it "more responsive to my needs" and "better at knowing when I was stuck."

4. THREATS TO VALIDITY

- Internal Validity: The random assignment of participants to conditions mitigates selection bias. The use of parallel pre- and post-tests controls for test familiarity.
- External Validity: The study was conducted in a single domain (Python programming) with a specific demographic (university students). Generalizability to other domains and age groups requires further research.
- Construct Validity: Our reward function, while multi-faceted, is still a proxy for the complex construct of "learning." The DKVMN model's mastery values are an approximation of true knowledge.

VII. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and empirical validation of RL-Tutor, an Intelligent Tutoring System that leverages Deep Reinforcement Learning for personalized skill development. Our contributions are threefold. First, we proposed a novel architecture that integrates a deep knowledge-tracing student model (DKVMN) with a stable, policy-based RL agent (PPO) to form a cohesive and adaptive tutoring loop. Second, we formulated a multi-faceted reward function that carefully balances immediate performance, learning efficiency, engagement, and, crucially, long-term knowledge retention, guiding the agent towards pedagogically sound strategies. Third, we demonstrated through large-scale simulated experiments and a human study that the resulting RL-based tutor significantly outperforms standard rule-based and static tutors in terms of learning gain, efficiency, and retention.

The learned policy exhibited sophisticated, emergent pedagogical behaviors, such as adaptive hint provisioning, strategic problem sequencing, and implicit spaced repetition, which were directly optimized for long-term student outcomes. This suggests that RL can indeed discover effective teaching strategies that may be non-obvious or too complex to hand-code.

Despite these promising results, several limitations and future directions remain. The current system operates in a bounded domain (Python programming); extending it to open-ended or ill-structured domains is a significant challenge. The reliance on a simulated environment for training, while necessary, introduces a reality gap; future work will involve fine-tuning the policy with real human interaction data using safe, off-policy RL methods. Furthermore, enhancing the interpretability of the RL policy is critical for gaining the trust of educators. We plan to develop visualization tools that explain the system's recommendations in terms of the student's inferred knowledge state and the predicted long-term benefit of a suggested action. Finally, integrating affective computing to model student engagement and frustration more directly could lead to even more nuanced and supportive pedagogical interactions. In conclusion, this work establishes a

strong foundation for the application of advanced RL techniques in creating truly personalized and effective learning environments.

REFERENCES

- [1] M. Benvenuti *et al.*, "Artificial intelligence and human behavioral development: A perspective on new skills and competences acquisition for the educational context," *Comput Human Behav*, vol. 148, 2023, doi: 10.1016/j.chb.2023.107903.
- [2] F. Niño-Rojas, D. Lancheros-Cuesta, M. T. P. Jiménez-Valderrama, G. Mestre, and S. Gómez, "Systematic Review: Trends in Intelligent Tutoring Systems in Mathematics Teaching and Learning," *International Journal of Education in Mathematics, Science and Technology*, vol. 12, no. 1, 2023, doi: 10.46328/ijemst.3189.
- [3] G. N. Vivekananda *et al.*, "Retracing-efficient IoT model for identifying the skin-related tags using automatic lumen detection," *Intelligent Data Analysis*, vol. 27, pp. 161–180, 2023, doi: 10.3233/IDA-237442.
- [4] J. A. Esponda-Pérez, M. A. Mousse, S. M. Almufti, I. Haris, S. Erdanova, and R. Tsarev, "Applying Multiple Regression to Evaluate Academic Performance of Students in E-Learning," 2024, pp. 227–235. doi: 10.1007/978-3-031-70595-3_24.
- [5] J. A. Esponda-Pérez *et al.*, "Application of Chi-Square Test in E-learning to Assess the Association Between Variables," 2024, pp. 274–281. doi: 10.1007/978-3-031-70595-3_28.
- [6] P. H. Nguyen, S. M. Almufti, J. A. Esponda-Pérez, D. Salguero García, I. Haris, and R. Tsarev, "The Impact of E-Learning on the Processes of Learning and Memorization," 2024, pp. 218–226. doi: 10.1007/978-3-031-70595-3_23.
- [7] A. Shaban, R. Rajab Asaad, and S. Almufti, "The Evolution of Metaheuristics: From Classical to Intelligent Hybrid Frameworks," *Qubahan Techno Journal*, vol. 1, no. 1, pp. 1–15, Jan. 2022, doi: 10.48161/qtj.v1n1a13.
- [8] R. Asaad, R. Ismail Ali, and S. Almufti, "Hybrid Big Data Analytics: Integrating Structured and Unstructured Data for Predictive Intelligence," *Qubahan Techno Journal*, vol. 1, no. 2, Apr. 2022, doi: 10.48161/qtj.v1n2a14.
- [9] M. K. Sharma, H. A. Alkhazaleh, S. Askar, N. H. Haroon, S. M. Almufti, and M. R. Al Nasar, "FEM-supported machine learning for residual stress and cutting force analysis in micro end milling of aluminum alloys," *International Journal of Mechanics and Materials in Design*, vol. 20, no. 5, pp. 1077–1098, Oct. 2024, doi: 10.1007/s10999-024-09713-9.
- [10] S. M. Abdulrahman, R. R. Asaad, H. B. Ahmad, A. Alaa Hani, S. R. M. Zeebaree, and A. B. Sallow, "Machine Learning in Nonlinear Material Physics," *Journal of Soft Computing and Data Mining*, vol. 5, no. 1, Jun. 2024, doi: 10.30880/jscdm.2024.05.01.010.
- [11] K. Rustamov, "5G-Enabled Internet of Things: Latency Optimization through AI-Assisted Network Slicing," *Qubahan Techno Journal*, vol. 2, no. 1, pp. 1–10, Feb. 2023, doi: 10.48161/qtj.v2n1a18.
- [12] N. Rustamova and , Raveenthiran Vivekanantharasa, "Comprehensive Review and Hybrid Evolution of Teaching–Learning–Based Optimization," *Qubahan Techno Journal*, vol. 2, no. 2, pp. 1–13, May 2023, doi: 10.48161/qtj.v2n2a19.
- [13] A. B. Sallow, R. R. Asaad, H. B. Ahmad, S. Mohammed Abdulrahman, A. A. Hani, and S. R. M. Zeebaree, "Machine Learning Skills To K–12," *Journal of Soft Computing and Data Mining*, vol. 5, no. 1, Jun. 2024, doi: 10.30880/jscdm.2024.05.01.011.
- [14] H. B. Ahmad, R. R. Asaad, S. M. Almufti, A. A. Hani, A. B. Sallow, and S. R. M. Zeebaree, "SMART HOME ENERGY SAVING WITH BIG DATA AND MACHINE LEARNING," *Jurnal Ilmiah Ilmu Terapan Universitas Jambi*, vol. 8, no. 1, pp. 11–20, May 2024, doi: 10.22437/jiituj.v8i1.32598.
- [15] D. A. Majeed *et al.*, "DATA ANALYSIS AND MACHINE LEARNING APPLICATIONS IN ENVIRONMENTAL MANAGEMENT," *Jurnal Ilmiah Ilmu Terapan Universitas Jambi*, vol. 8, no. 2, pp. 398–408, Sep. 2024, doi: 10.22437/jiituj.v8i2.32769.
- [16] D. A. Majeed *et al.*, "DATA ANALYSIS AND MACHINE LEARNING APPLICATIONS IN ENVIRONMENTAL MANAGEMENT," *Jurnal Ilmiah Ilmu Terapan Universitas Jambi*, vol. 8, no. 2, pp. 398–408, Sep. 2024, doi: 10.22437/jiituj.v8i2.32769.

-
- [17] UU Republik Indonesia *et al.*, "PENENTUAN ALTERNATIF LOKASI TEMPAT PEMBUANGAN AKHIR (TPA) SAMPAH DI KABUPATEN SIDOARJO," *Energies (Basel)*, vol. 15, no. 1, 2022.
 - [18] S. M. Almufti *et al.*, "INTELLIGENT HOME IOT DEVICES: AN EXPLORATION OF MACHINE LEARNING-BASED NETWORKED TRAFFIC INVESTIGATION," *Jurnal Ilmiah Ilmu Terapan Universitas Jambi*, vol. 8, no. 1, pp. 1–10, May 2024, doi: 10.22437/jiituj.v8i1.32767.
 - [19] A. Yahya, "Systematic Review of Regression Algorithms for Predictive Analytics," *Qubahan Techno Journal*, vol. 1, no. 4, Nov. 2022, doi: 10.48161/qtj.v1n4a17.
 - [20] J. A. Dela Fuente, "Automated Software Testing through Large Language Models: Opportunities and Challenges," *Qubahan Techno Journal*, vol. 1, no. 3, pp. 1–16, Jul. 2022, doi: 10.48161/qtj.v1n3a15.
 - [21] R. Rajab Asaad, R. Ismael Ali, Z. Arif Ali, and A. Ahmad Shaaban, "Image Processing with Python Libraries," *Academic Journal of Nawroz University*, vol. 12, no. 2, pp. 410–416, Jun. 2023, doi: 10.25007/ajnu.v12n2a1754.
 - [22] Ç. Sıcakyüz, R. Rajab Asaad, S. Almufti, and N. R. Rustamova, "Adaptive Deep Learning Architectures for Real-Time Data Streams in Edge Computing Environments," *Qubahan Techno Journal*, vol. 3, no. 2, pp. 1–14, Jun. 2024, doi: 10.48161/qtj.v3n2a25.
 - [23] Z. Liu, P. Agrawal, S. Singhal, V. Madaan, M. Kumar, and P. K. Verma, "LPITutor: An LLM based personalized intelligent tutoring system using RAG and prompt engineering," *PeerJ Comput Sci*, vol. 11, 2025, doi: 10.7717/peerj-cs.2991.
 - [24] A. Ahmed Shaban, S. M. Almufti, and R. B. Marqas, "A Modified Bat Algorithm for Economic Dispatch with Enhanced Performance Metrics," *FMDb Transactions on Sustainable Technoprise Letters*, vol. 3, no. 2, pp. 59–72, Jun. 2025, doi: 10.69888/ftstpl.2025.000437.
 - [25] S. M. Almufti, R. B. Marqas, Z. A. Nayef, and T. S. Mohamed, "Real Time Face-mask Detection with Arduino to Prevent COVID-19 Spreading," *Qubahan Academic Journal*, vol. 1, no. 2, pp. 39–46, Apr. 2021, doi: 10.48161/qaj.v1n2a47.
 - [26] S. M. Almufti and A. M. Abdulazeez, "An Integrated Gesture Framework of Smart Entry Based on Arduino and Random Forest Classifier," *Indonesian Journal of Computer Science*, vol. 13, no. 1, Feb. 2024, doi: 10.33022/ijcs.v13i1.3735.